

6장 합성곱 신경망 2 – part2 (6.1.5~)

2팀 곽채원 이효주 한수연

EWHA EURON

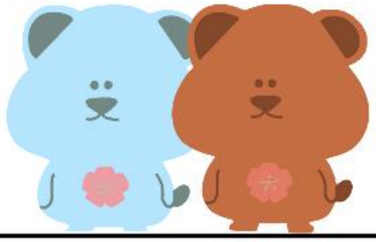
목차

6.1 이미지 분류를 위한 신경망

6.1.5 ResNet

6.2 객체 인식을 위한 신경망

6.3 이미지 분할을 위한 신경망



EWHA EURON

6.1.5 ResNet



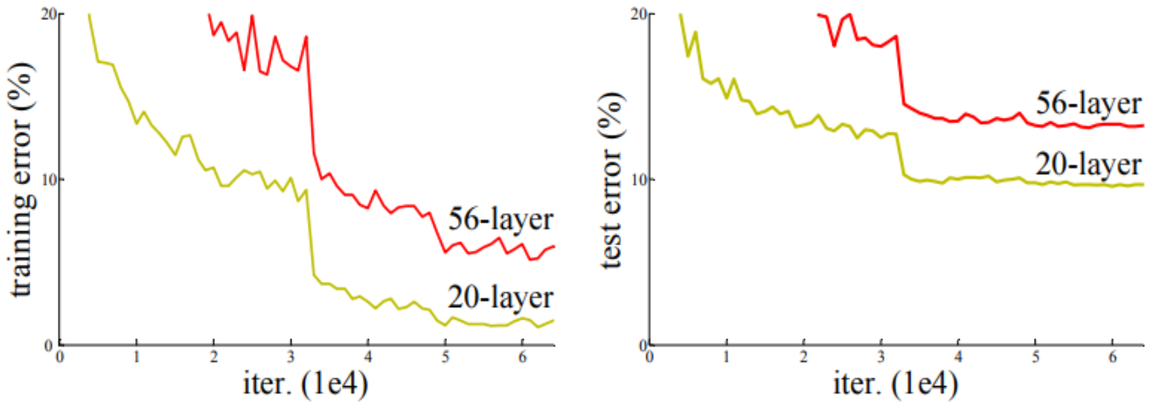
#6.1.5 ResNet

등장 배경

신경망의 깊이가 깊어질수록 성능이 좋아지다가 일정한 단계에 다다르면 성능 저하 발생

이는 신경망이 깊어지면 최적화에서 low convergence rate가 매우 낮기 때문에 발생하는 문제

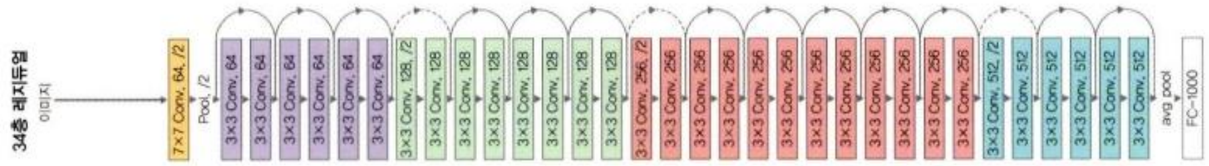
› **최적화를 쉽게 하는 Residual Block 도입 !**



#6.1.5 ResNet

용어 설명

1. **블록** : 계층의 묶음
2. **아이덴티티 매핑(shortcut, skip connection)** : 입력x가 어떤 함수를 통과하더라도 다시 x로 출력
3. **다운 샘플** : residual block 간 형태를 맞춤




```
def forward(self, x) :
    i = x    # 입력x를 i 변수에 저장
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.conv2(x)
    x = self.bn2=(x)

    if self.downsample is not None :
        i = self.downsample(i) # 다운 샘플 적용
    x += I # 아이덴티티 매핑 적용
    x = self.relu(x)
    return x
```

#6.1.5 ResNet

구조

기존 방식

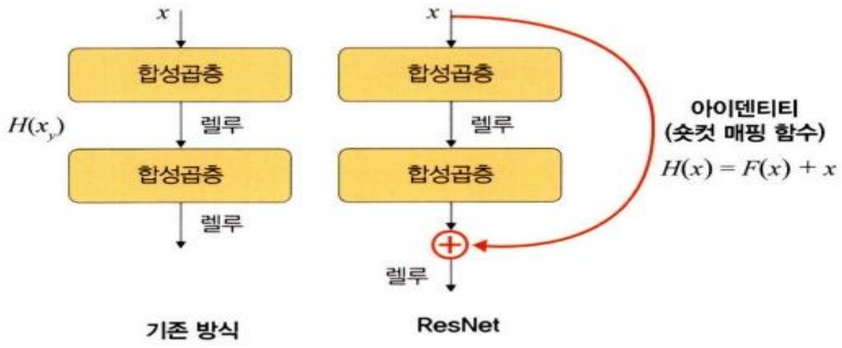
: input으로 x를 받아서 두 개의 weight layer를 거치고, 학습을 통해 최적의 출력 H(x)를 학습

ResNet

: H(x)만을 출력하는 기존의 방식에서 출력인 H(x)와 입력인 x의 차이를 F(x)라 두고, 이를 학습

즉 residual function인 F(x) = H(x) – x를 최소화해야 하고, 즉 출력과 입력의 차를 줄이는 것

F(x)=0이 최적의 해이므로, H(x)=x로 매핑하는 것이 학습의 목표



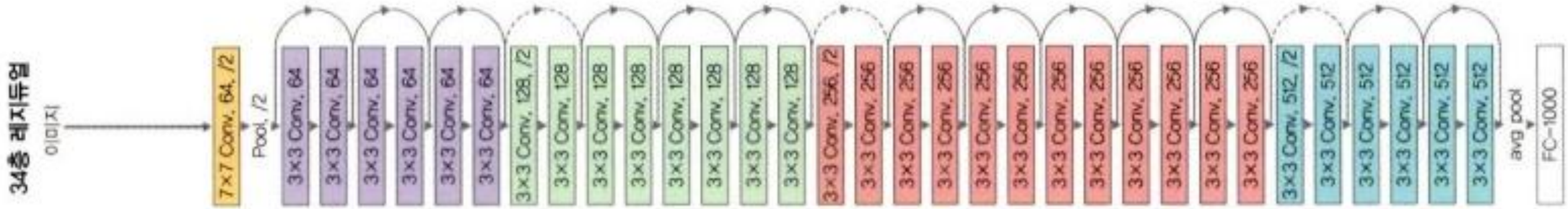
#6.1.5 ResNet

구조

Identity Shortcut $\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}.$

Projection Shortcut $\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}.$

Ex. 점선으로 표시된 부분에서 output의 개수가 두배로 튀지만, 가로 세로 길이는 절반으로 줄어들어 차원이 일치하지 않은 경우



#6.1.5 ResNet

구조

ResNet은 기본적으로VGG19 구조를 뼈대로 하여 shortcut들을 추가하는 것

다만 역시 계층이 깊어질수록 파라미터의 수가 증가하는 문제 발생…

› **Bottleneck Block도입 !**

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9



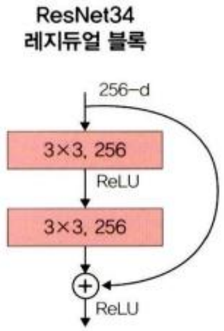
#6.1.5 ResNet

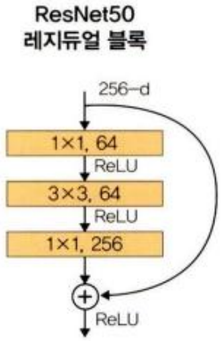
Bottleneck Block

: residual block의 일종으로 연산량을 줄이기 위해 1x1 합성곱을 추가한 것

: 처음에 1x1 연산으로 특성 맵의 개수를 줄이고, 줄어든 특성맵의 개수로 3x3 연산을 수행 후, 다시 차원을 키움

: layer의 개수가 50개 이상인 경우부터 사용





	기본 블록	병목 블록
ResNet	ResNet18, ResNet34	ResNet50, ResNet101, ResNet152
구조	3x3 합성곱 두 개	1x1 -> 3x3 -> 1x1 합성곱
파라미터 수	39.321M (ResNet34)	6.9632M (ResNet50)

#6.1.5 ResNet

실습

라이브러리 호출

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as datasets
import torchvision.models as models

import matplotlib.pyplot as plt
import numpy as np

import copy
from collections import namedtuple
import os
import random
import time

import cv2
from torch.utils.data import DataLoader, Dataset
from PIL import Image

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

데이터 전처리

```
# 코드 6-66 이미지 데이터 전처리

class ImageTransform() :
  def __init__(self, resize, mean, std) :
    self.data_transform = {
        'train' : transforms.Compose([
            transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),
            transforms.RandomHorizontalFlip(),
            transforms.ToTensor(),
            transforms.Normalize(mean, std)
        ]),
        'val' : transforms.Compose([
            transforms.Resize(256),
            transforms.CenterCrop(resize),
            transforms.ToTensor(),
            transforms.Normalize(mean, std)
        ])
    }

  def __call__(self, img, phase) :
    return self.data_transform[phase](img)
```

```
# 코드 6-67 변수에 대한 값 정의

size = 224
mean =(0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

EWHA EURON

#6.1.5 ResNet

실습

데이터 준비

```
# API 토큰 설정
os.environ['KAGGLE_API_TOKEN'] = "KGAT_e8e62cb7bb85abd8d25748922bf40184"

# 공개 데이터셋 다운로드
!kaggle datasets download -d chetankv/dogs-cats-images

# dataset 폴더에 압축 해제
!unzip -q dogs-cats-images.zip -d ./dataset
```

데이터 분리

```
# 코드 6-69 데이터셋을 훈련, 검증, 테스트 용도로 분리
random.seed(42)
random.shuffle(correct_images_filepaths)


train_images_filepaths = correct_images_filepaths[:7000]
val_images_filepaths = correct_images_filepaths[7000:-1000]
test_images_filepaths = correct_images_filepaths[-1000:]

print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))
```

```
7000 2000 1000
```

데이터 불러오기

```
# 코드 6-68 훈련과 테스트 데이터셋 불러오기

cat_directories = [r'./dataset/dataset/training_set/cats/',r'./dataset/dataset/test_set/cats/']
dog_directories = [r'./dataset/dataset/training_set/dogs/',r'./dataset/dataset/test_set/dogs/']

cat_images_filepaths = sorted([os.path.join(d, f) for d in cat_directories for f in os.listdir(d)])
dog_images_filepaths = sorted([os.path.join(d, f) for d in dog_directories for f in os.listdir(d)])

images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]

print(len(correct_images_filepaths))
```

EWHA EURON

#6.1.5 ResNet

실습

레이블 구분

```
# 코드 6-70 레이블 구분

class DogsCatsDataset(Dataset):
  def __init__(self, file_list, transform=None, phase='train'):
    self.file_list = file_list
    self.transform = transform
    self.phase = phase

  def __len__(self):
    return len(self.file_list)

  def __getitem__(self, idx):
    img_path = self.file_list[idx]
    img = Image.open(img_path)
    img_transformed = self.transform(img, self.phase)

    label = img_path.split('/')[-1].split('.')[0]
    if label == 'dog' :
      label =1
    elif label == 'cat' :
      label = 0
    return img_transformed, label
```

데이터셋 정의

```
# 6-71 이미지와 데이터셋 정의

train_dataset = DogsCatsDataset(train_images_filepaths, transform=ImageTransform(size, mean, std), phase='train')
val_dataset = DogsCatsDataset(val_images_filepaths, transform=ImageTransform(size, mean, std), phase='val')

index = 0
print(train_dataset.__getitem__(index)[0].size())
print(train_dataset.__getitem__(index)[1])
```

```
torch.Size([3, 224, 224])
0
```

DataLoader로 메모리에 불러오기

```
# 코드 6-72 데이터셋의 데이터를 메모리로 불러오기

train_iterator = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_iterator = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_iterator, 'val': val_iterator}

batch_iterator = iter(train_iterator)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)
```

```
torch.Size([32, 3, 224, 224])
tensor([1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0,
        0, 1, 1, 0, 0, 0, 0, 1])
```

EWHA EURON

6.1.5 ResNet (2)



#6.1.5 ResNet (2)

ResNet 구현 순서

1. **블록 정의**
   기본 블록 (BasicBlock) → 병목 블록 (Bottleneck)

2. **네트워크 정의**
   ResNet 클래스 → ResNetConfig → 모델별 Config 설정

3. **학습 준비**
   옵티마이저 · 손실 함수 → 정확도 측정 함수

4. **학습과 평가**
   train → evaluate → 모델 학습 → 테스트 예측

앞 파트에서 다룬 내용
ResNet 의 필요성과 탄생 배경
레지듀얼 블록과 스킵 연결
병목 블록, 아이덴티티 매핑
데이터 준비 및 전처리

이번 파트
위 내용을 실제 파이토치 코드로 어떻게 구현하는지 살펴봄

EWHA EURON

#1. 기본 블록 (BasicBlock)

기본 블록 정의

```
class BasicBlock(nn.Module):
    expansion = 1

    def __init__(self, in_channels, out_channels,
                 stride=1, downsample=False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,
                               stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                               stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)

        if downsample:
            conv = nn.Conv2d(in_channels, out_channels, kernel_size=1,
                             stride=stride, bias=False)
            bn = nn.BatchNorm2d(out_channels)
            downsample = nn.Sequential(conv, bn)
        else:
            downsample = None
        self.downsample = downsample
```

기본 블록은 ResNet18, ResNet34 에서 사용되며 합성곱 (3 x 3) 두 개로 구성됨

conv1, conv2 → 3 x 3 합성곱층

각 합성곱층 뒤에 배치 정규화 (BatchNorm2d)

① 다운샘플 (downsample)

입력 데이터의 크기와 네트워크를 통과한 후 출력 데이터의 크기가 다를 경우에 사용

다운샘플을 위해서는 합성곱층에 스트라이드를 적용

EWHA EURON

#1. 기본 블록 (BasicBlock)

forward — 아이덴티티 매핑

```
def forward(self, x):
    i = x
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.conv2(x)
    x = self.bn2(x)

    if self.downsample is not None:
        i = self.downsample(i)

    x += i
    x = self.relu(x)

    return x
```

② 아이덴티티 매핑 (identity mapping)

특정 층에 존재하는 출력 결과를 다음 합성곱층을 통과한 출력 결과에 더해 준다고 하여 스킵 연결 (skip connection) 이라고도 함

x 에 conv1, bn1, relu, conv2, bn2, relu, conv3, bn3 값이 더해지다가 초기의 x 가 다시 더해지는 것

맨 위 i = x 로 입력을 저장해 두고

맨 아래 x += i 로 다시 더해 줌

EWHA EURON

#2. 병목 블록 (Bottleneck)

병목 블록 정의

```
class Bottleneck(nn.Module):
    expansion = 4

    def __init__(self, in_channels, out_channels,
                 stride=1, downsample=False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=1,
                               stride=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                               stride=stride, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.conv3 = nn.Conv2d(out_channels, self.expansion*out_channels,
                               kernel_size=1, stride=1, bias=False)
        self.bn3 = nn.BatchNorm2d(self.expansion*out_channels)
        self.relu = nn.ReLU(inplace=True)

        if downsample:
            conv = nn.Conv2d(in_channels, self.expansion*out_channels,
                             kernel_size=1, stride=stride, bias=False)
            bn = nn.BatchNorm2d(self.expansion*out_channels)
            downsample = nn.Sequential(conv, bn)
        else:
            downsample = None
        self.downsample = downsample
```

병목 블록은 ResNet50, ResNet101, ResNet152 에서 사용되며 1 x 1 합성곱층 , 3 x 3 합성곱층 , 1 x 1 합성곱층으로 구성됨

expansion = 4

ResNet 에서 병목 블록을 정의하기 위한 하이퍼파라미터

conv3 의 self.expansion*out_channels

다음 계층의 입력 채널 수와 일치하도록 맞춰 줌

EWHA EURON

#2. 병목 블록 (Bottleneck)

기본 블록 vs 병목 블록

기본 블록 (BasicBlock)

3 × 3 합성곱층 두 개

- ResNet18, ResNet34 에서 사용
- expansion = 1

병목 블록 (Bottleneck)

1 × 1, 3 × 3, 1 × 1 합성곱층

- ResNet50, ResNet101, ResNet152 에서 사용
- expansion = 4
- 계층을 더 깊게 쌓으면서 계산에 대한 비용을 줄일 수 있음
- 계층이 많아진다는 것은 곧 활성화 함수가 기존보다 더 많이 포함된다는 것
- → 더 많은 비선형성 (non-linearity) 을 처리할 수 있음
- → 즉, 다양한 입력 데이터에 대한 처리가 가능

결국 아이덴티티 매핑과 병목 블록으로 ResNet 네트워크에 더욱 깊은 계층을 쌓을 수 있게 된 것

EWHA EURON

#3. ResNet 모델 네트워크

계층 쌓기 — conv1, maxpool, layer1~layer4

```
class ResNet(nn.Module):
    def __init__(self, config, output_dim, zero_init_residual=False):
        super().__init__()

        block, n_blocks, channels = config
        self.in_channels = channels[0]
        assert len(n_blocks) == len(channels) == 4

        self.conv1 = nn.Conv2d(3, self.in_channels, kernel_size=7,
                               stride=2, padding=3, bias=False)
        self.bn1 = nn.BatchNorm2d(self.in_channels)
        self.relu = nn.ReLU(inplace=True)
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)

        self.layer1 = self.get_resnet_layer(block, n_blocks[0], channels[0])
        self.layer2 = self.get_resnet_layer(block, n_blocks[1], channels[1], stride=2)
        self.layer3 = self.get_resnet_layer(block, n_blocks[2], channels[2], stride=2)
        self.layer4 = self.get_resnet_layer(block, n_blocks[3], channels[3], stride=2)

        self.avgpool = nn.AdaptiveAvgPool2d((1,1))
        self.fc = nn.Linear(self.in_channels, output_dim)
```

ResNet 을 호출할 때 넘겨준 config 값들을 block, n_blocks, channels 에 저장

assert — 블록 크기 = 채널 크기 = 4

layer1~layer4 는 get_resnet_layer 함수로 블록을 쌓음

layer2~layer4 에만 stride=2 적용

EWHA EURON

#3. ResNet 모델 네트워크

get_resnet_layer — 블록을 쌓는 함수

```
def get_resnet_layer(self, block, n_blocks, channels, stride=1):
    layers = []
    if self.in_channels != block.expansion * channels:
        downsample = True
    else:
        downsample = False

    layers.append(block(self.in_channels, channels, stride, downsample))
    for i in range(1, n_blocks):
        layers.append(block(block.expansion*channels, channels))

    self.in_channels = block.expansion * channels
    return nn.Sequential(*layers)
```

블록을 추가하기 위한 함수

in_channels 와 block.expansion*channels 가 다르면 downsample 적용

계층 (layer) 을 추가할 때 in_channels, channels, stride 뿐만 아니라
다운샘플 적용 유무도 함께 전달

for 문으로 n_blocks 만큼 계층 추가

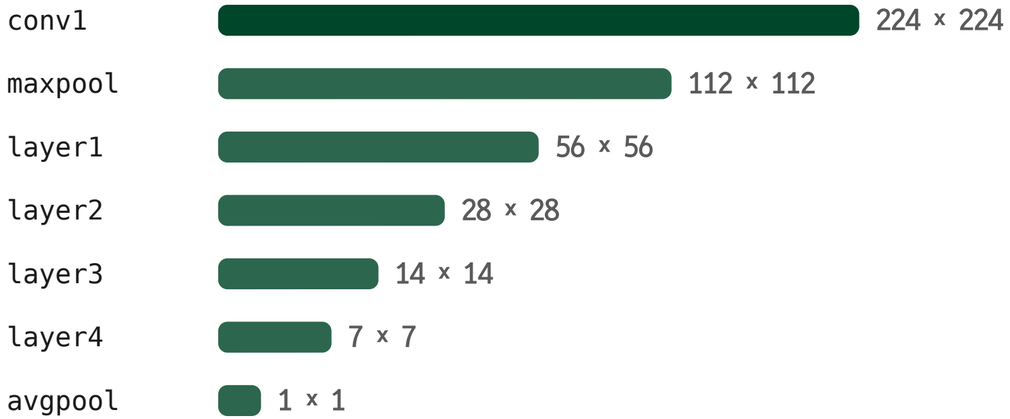
EWHA EURON

#3. ResNet 모델 네트워크

forward

```
def forward(self, x):
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.maxpool(x)
    x = self.layer1(x)
    x = self.layer2(x)
    x = self.layer3(x)
    x = self.layer4(x)
    x = self.avgpool(x)
    h = x.view(x.shape[0], -1)
    x = self.fc(h)
    return x, h
```

층별 데이터 크기 변화



층을 지날수록 데이터의 크기가 줄어듦

EWHA EURON

#3. ResNet 모델 네트워크

zero_init_residual

```
if zero_init_residual:
    for m in self.modules():
        if isinstance(m, Bottleneck):
            nn.init.constant_(m.bn3.weight, 0)
        elif isinstance(m, BasicBlock):
            nn.init.constant_(m.bn2.weight, 0)
```

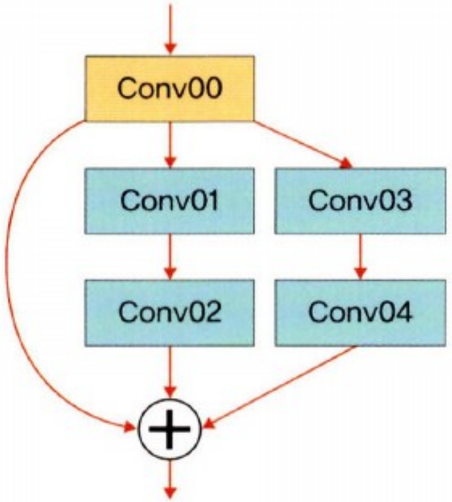
각 레지듀얼 분기 (residual branch) 에 있는 마지막 BN(Batch Normalization) 을 0 으로 초기화해서 다음 레지듀얼 분기를 0 에서 시작할 수 있도록 함

모델을 생성하고 학습시키는 것과는 상관없음

논문에 의하면 BN 을 0 으로 초기화할 경우 모델 성능이 0.2~0.3% 정도 향상

그림 6-33 레지듀얼 분기

그림 6-33 레지듀얼 분기



레지듀얼 분기란 프로그램에서의 조건에 따라 A, B, C 등으로 분기하는 것과 같음

EWHA EURON

#4. ResNetConfig 정의

ResNetConfig 정의

```
ResNetConfig = namedtuple('ResNetConfig',
                          ['block', 'n_blocks', 'channels'])
```

ResNetConfig 변수에 네임드튜플 데이터 형식으로 ['block', 'n_blocks', 'channels'] 를 저장

기본 블록을 사용하는 Config

```
resnet18_config = ResNetConfig(block=BasicBlock,
                               n_blocks=[2,2,2,2],
                               channels=[64,128,256,512])

resnet34_config = ResNetConfig(block=BasicBlock,
                               n_blocks=[3,4,6,3],
                               channels=[64,128,256,512])
```

기본 블록을 사용하는 ResNet18 과 ResNet34 의 Config 를 정의

블록은 기본 블록 (BasicBlock) 을 사용하도록 하며 , 블록 (n_blocks) 과 채널 (channels) 의 크기를 각각 지정

ResNet18 [2, 2, 2, 2]

ResNet34 [3, 4, 6, 3]

EWHA EURON

#4. ResNetConfig 정의

병목 블록을 사용하는 Config

```
resnet50_config = ResNetConfig(block=Bottleneck,
                               n_blocks=[3,4,6,3],
                               channels=[64,128,256,512])

resnet101_config = ResNetConfig(block=Bottleneck,
                                n_blocks=[3,4,23,3],
                                channels=[64,128,256,512])

resnet152_config = ResNetConfig(block=Bottleneck,
                                n_blocks=[3,8,36,3],
                                channels=[64,128,256,512])
```

모델별 설정 정리

모델	block	n_blocks
ResNet50	Bottleneck	[3, 4, 6, 3]
ResNet101	Bottleneck	[3, 4, 23, 3]
ResNet152	Bottleneck	[3, 8, 36, 3]

블록은 병목 블록 (Bottleneck) 을 사용하도록 하며 , 블록 (n_blocks) 과 채널 (channels) 의 크기를 각각 지정

channels 는 모두 [64, 128, 256, 512]

EWHA EURON

#5. 사전 훈련된 ResNet과 비교

사전 훈련된 ResNet 모델 사용

```
pretrained_model = models.resnet50(pretrained=True)
```

사전 훈련된 ResNet 모델 사용을 위해서는 pretrained=True로 설정

ResNet 역시 사전 훈련된 모델이기 때문에 코드 한 줄로 간단하게 사용할 수 있음

힘들게 정의한 네트워크와 사전 정의된 네트워크가 다르지 않음

직접 만든 ResNet 사용

```
OUTPUT_DIM = 2
model = ResNet(resnet50_config, OUTPUT_DIM)
print(model)
```

OUTPUT_DIM에는 테스트 데이터셋의 클래스(개와 고양이) 수를 입력

그 구조를 알고 사용하는 것과 모르고 사용하는 것은 큰 차이가 있음

새로운 모델의 네트워크가 발표되더라도 그 구조부터 익히는 훈련이 필요

EWHA EURON

#6. 학습 준비

옵티마이저와 손실 함수 정의

```
optimizer = optim.Adam(model.parameters(), lr=1e-7)
criterion = nn.CrossEntropyLoss()

model = model.to(device)
criterion = criterion.to(device)
```

lr=1e-7 은 1*10 의 -7 승을 의미

모델과 손실 함수를 device 로 보냄

모델 학습 시간 측정 함수 정의

```
def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time/60)
    elapsed_secs = int(elapsed_time-(elapsed_mins*60))
    return elapsed_mins, elapsed_secs
```

모델을 학습시키는 데 어느 정도의 시간이 걸리는지 측정하기 위한 함수

EWHA EURON

#6. 학습 준비

모델 학습 정확도 측정 함수 정의

```
def calculate_topk_accuracy(y_pred, y, k=2):
    with torch.no_grad():
        batch_size = y.shape[0]
        _, top_pred = y_pred.topk(k, 1)
        top_pred = top_pred.t()
        correct = top_pred.eq(y.view(1, -1).expand_as(top_pred))
        correct_1 = correct[:1].reshape(-1).float().sum(0, keepdim=True)
        correct_k = correct[:k].reshape(-1).float().sum(0, keepdim=True)
        acc_1 = correct_1 / batch_size
        acc_k = correct_k / batch_size
    return acc_1, acc_k
```

① tensor.topk 는 torch.argmax 와 같은 효과

주어진 텐서에서 가장 큰 값의 인덱스를 얻기 위해 사용. 즉, 네트워크의 출력에서 가장 확률이 높은 값의 인덱스를 반환

② t() 는 차원 0 과 1 을 전치 (transpose) 하겠다는 의미

③ 텐서를 비교하는 함수

torch.eq 서로 같은지 / torch.ne 서로 다른지 / torch.ge 크거나 같은지

EWHA EURON

#7. 학습 함수

모델 학습 함수 정의

```python
def train(model, iterator, optimizer, criterion, scheduler, device):
    epoch_loss = 0
    epoch_acc_1 = 0
    epoch_acc_5 = 0

    model.train()
    for (x, y) in iterator:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()
        y_pred = model(x)
        loss = criterion(y_pred[0], y)

        acc_1, acc_5 = calculate_topk_accuracy(y_pred[0], y)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()
        epoch_acc_1 += acc_1.item()
        epoch_acc_5 += acc_5.item()

    epoch_loss /= len(iterator)
    epoch_acc_1 /= len(iterator)
    epoch_acc_5 /= len(iterator)
    return epoch_loss, epoch_acc_1, epoch_acc_5
```

model.train() 으로 학습 모드 설정

zero_grad() → 예측 → 손실 계산 → backward() → step()

epoch_acc_1

모델이 첫 번째로 예측한 레이블이 붙여짐

epoch_acc_5

이미지에 정확한 레이블이 붙여질 것이기 때문에 정확도가 100% 일 것

EWHA EURON

#7. 평가 함수

모델 평가 함수 정의

```
def evaluate(model, iterator, criterion, device):
    epoch_loss = 0
    epoch_acc_1 = 0
    epoch_acc_5 = 0

    model.eval()
    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y = y.to(device)
            y_pred = model(x)
            loss = criterion(y_pred[0], y)

            acc_1, acc_5 = calculate_topk_accuracy(y_pred[0], y)
            epoch_loss += loss.item()
            epoch_acc_1 += acc_1.item()
            epoch_acc_5 += acc_5.item()

    epoch_loss /= len(iterator)
    epoch_acc_1 /= len(iterator)
    epoch_acc_5 /= len(iterator)
    return epoch_loss, epoch_acc_1, epoch_acc_5
```

train 함수와의 차이

구분	train	evaluate
모드	model.train()	model.eval()
기울기	계산함	torch.no_grad()
역전파	loss.backward()	없음
가중치 갱신	optimizer.step()	없음

나머지 구조는 train 함수와 동일

EWHA EURON

#8. 모델 학습

모델 학습

```
best_valid_loss = float('inf')
EPOCHS = 10

for epoch in range(EPOCHS):
    start_time = time.monotonic()

    train_loss, train_acc_1, train_acc_5 = train(model, train_iterator,
                           optimizer, criterion, scheduler, device)
    valid_loss, valid_acc_1, valid_acc_5 = evaluate(model, valid_iterator,
                           criterion, device)

    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), '../chap06/data/ResNet-model.pt')

    end_time = time.monotonic()
    epoch_mins, epoch_secs = epoch_time(start_time, end_time)

    print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
    ...   # Train/Valid Loss, Acc  출력
```

출력 결과

```
Epoch: 01 | Epoch Time: 4m 4s
  Train Loss: 0.698 | Train Acc @1: 50.24% | Train Acc @5: 100.00%
  Valid Loss: 0.699 | Valid Acc @1: 51.19% | Valid Acc @5: 100.00%
            ...
Epoch: 06 | Epoch Time: 4m 3s
  Train Loss: 0.686 | Train Acc @1: 58.41% | Train Acc @5: 100.00%
  Valid Loss: 0.693 | Valid Acc @1: 49.85% | Valid Acc @5: 100.00%
            ...
Epoch: 10 | Epoch Time: 4m 1s
  Train Loss: 0.690 | Train Acc @1: 57.69% | Train Acc @5: 100.00%
  Valid Loss: 0.693 | Valid Acc @1: 51.93% | Valid Acc @5: 100.00%
```

0.690
Train Loss

57.69%
Train Acc @1

51.93%
Valid Acc @1

모델 학습 결과에 대한 오차와 정확도 측면에서 성능이 좋지 않음

이미지 데이터를 늘린다면 성능은 좋아질 수 있음

이 예제의 목적은 성능 향상이 아닌 CNN 관련 네트워크의 사용 방법이므로, 빠른 학습을 위해 데이터 개수를 제한시켰기 때문

#8. 테스트 예측

테스트 데이터셋을 이용한 모델 예측

```
import pandas as pd
id_list = []
pred_list = []
_id = 0
with torch.no_grad():
    for test_path in test_images_filepaths:
        img = Image.open(test_path)
        _id = test_path.split('/')[-1].split('.')[1]
        transform = ImageTransform(size, mean, std)
        img = transform(img, phase='val')
        img = img.unsqueeze(0)
        img = img.to(device)

        model.eval()
        outputs = model(img)
        preds = F.softmax(outputs[0], dim=1)[:, 1].tolist()
        id_list.append(_id)
        pred_list.append(preds[0])

res = pd.DataFrame({'id': id_list, 'label': pred_list})

res.sort_values(by='id', inplace=True)
res.reset_index(drop=True, inplace=True)

res.to_csv('../chap06/data/ResNet.csv', index=False)
res.head(10)
```

그림 6-34 예측 결과

그림 6-34 테스트 데이터셋에 대한 ResNet 모델의 예측 결과



테스트 데이터셋으로 예측한 결과를 ResNet.csv 로 저장

레이블 (label) 이 0.5 보다 크면 개 , 0.5 보다 작으면 고양이를 의미

역시 정확도가 높지 않음 – 성능을 향상시키려면 데이터를 더 늘려야 함

성능 향상 관련해서는 8 장을 참고

EWHA EURON

6.2 객체 인식을 위한 신경망



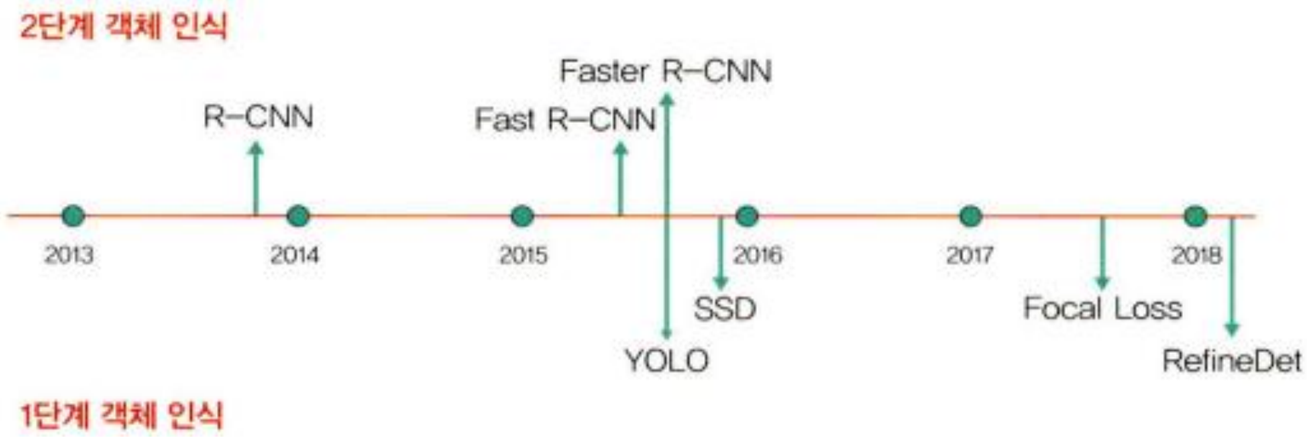
EWHA EURON

#01. 객체 인식

객체인식
= 여러 가지 객체에 대한 분류 + 객체의 위치 정보를 파악하는 위치 검출
-> 위치는 bounding box(바운딩 박스)로 표현

1단계 방식
: 분류와 위치 검출을 동시에 수행
➔ 빠르지만 정확도는 상대적으로 낮음
Ex. YOLO, SSD

2단계 방식
: 위치 검출 -> 분류를 순차적으로 수행
➔ 느리지만 정확도는 상대적으로 높음
Ex. R-CNN 계열



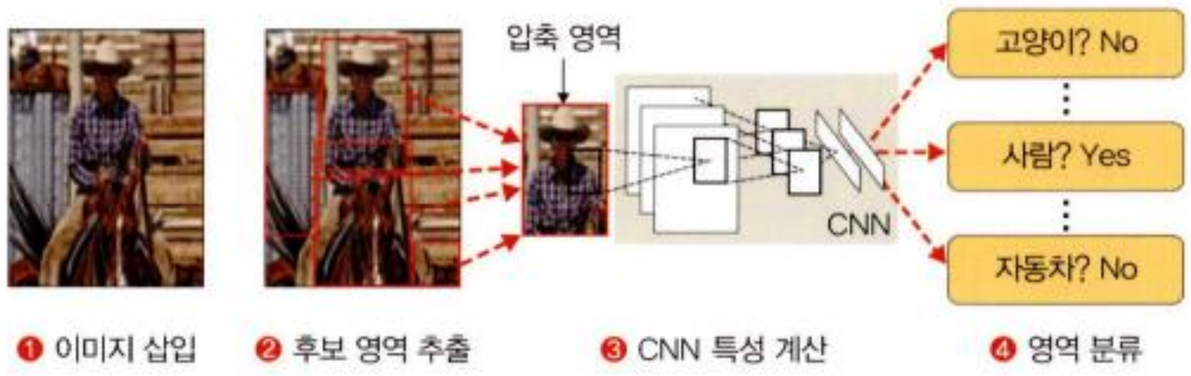
#02. R-CNN

기존 sliding window 방식
: 이미지 전체를 반복 탐색 -> 비효율적

region proposal(후보 영역)
: 객체가 있을 법한 영역만 먼저 추림

R-CNN : 후보 영역 알고리즘 + 이미지 분류용 CNN

R-CNN 학습 절차



1. 이미지를 입력으로 받음
2. 2000개의 바운딩 박스를 선택적 탐색 알고리즘으로 추출 ->crop->227x227픽셀로 통일
3. CNN 모델 적용
4. 분류 진행

EWHA EURON

#03. Selective search

선택적 탐색 단계

1단계(초기 영역 생성) : 이미지를 여러 개의 작은 초기 영역으로 세분화



#03. Selective search

선택적 탐색 단계

2단계 : greedy 알고리즘을 사용하여 비슷한 영역을 찾아 합침



#03. Selective search

선택적 탐색 단계

3단계 : 통합 영역을 기준으로 바운딩 박스를 만들고, 최종 후보 영역으로 사용

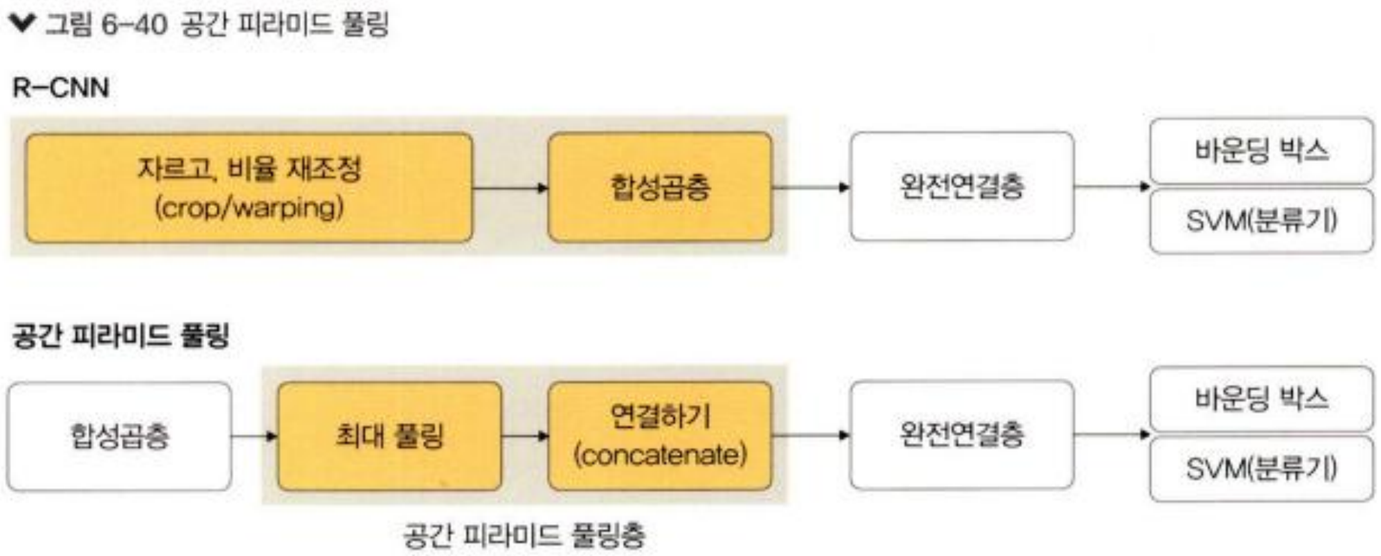


선택적 탐색을 통해 이미지 전체에서 객체가 존재할 가능성이 있는 위치를 좁혀줌

EWHA EURON

#04. R-CNN의 한계와 공간 피라미드 풀링

R-CNN의 한계
- 복잡한 학습 과정, 긴 학습 시간, 대용량 저장공간, 느린 객체 검출
- 기존 CNN의 완전연결층 -> 고정된 입력 크기가 필요
  - Crop/warp 과정에서 객체 일부가 잘리거나 모양이 변형 될 수 있음



공간 피라미드 풀링
: 원래 크기대로 CNN 통과 -> 특성 맵을 완전연결층에 넣기 직전에 동일 크기로 조절

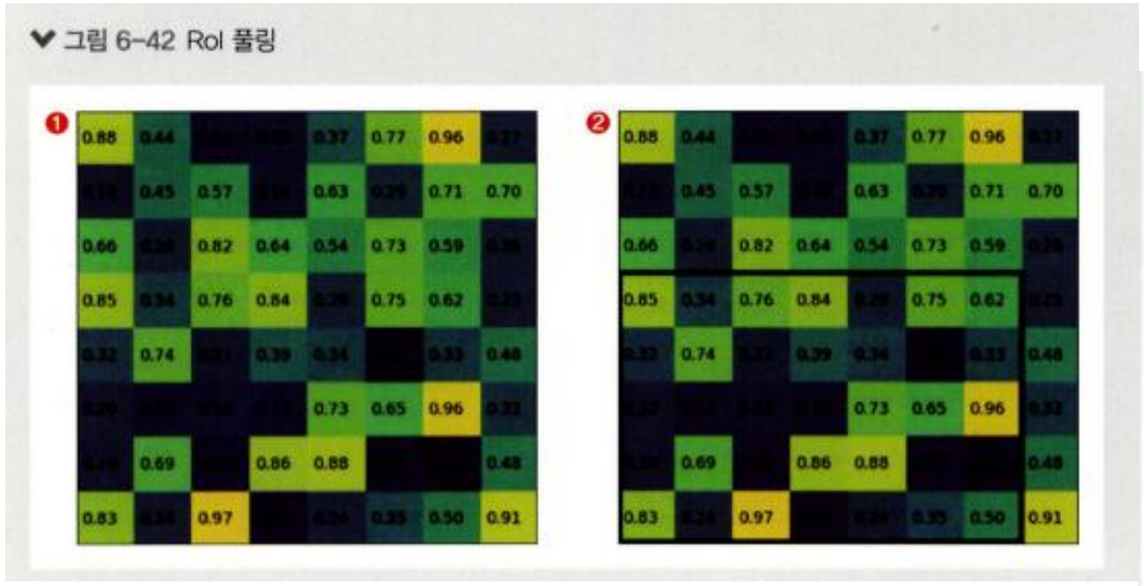
EWHA EURON

#05. Fast R-CNN

R-CNN : 후보 영역마다 CNN 반복

Fast R-CNN : 전체 이미지에 CNN 한번 -> 하나의 feature map 공유
- Selelctive search 후보 영역을 feature map 위에 대응

Region of Interest(= RoI) 풀링
: 서로 다른 크기의 RoI를 고정된 크기로 변환
=> 반복 합성곱 연산 감소 -> 속도 개선

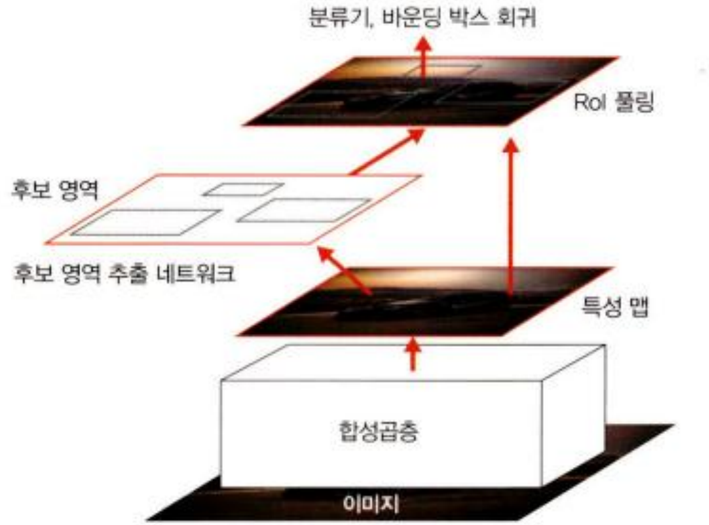


#06. Faster R-CNN

Fast R-CNN의 한계 : selective search

Faster R-CNN : 외부 후보 영역 알고리즘을 RPN(Region Proposal Network)으로 대체
➔ 후보 영역 생성 자체를 CNN 내부 네트워크로 가져옴

그림 6-43 Faster R-CNN

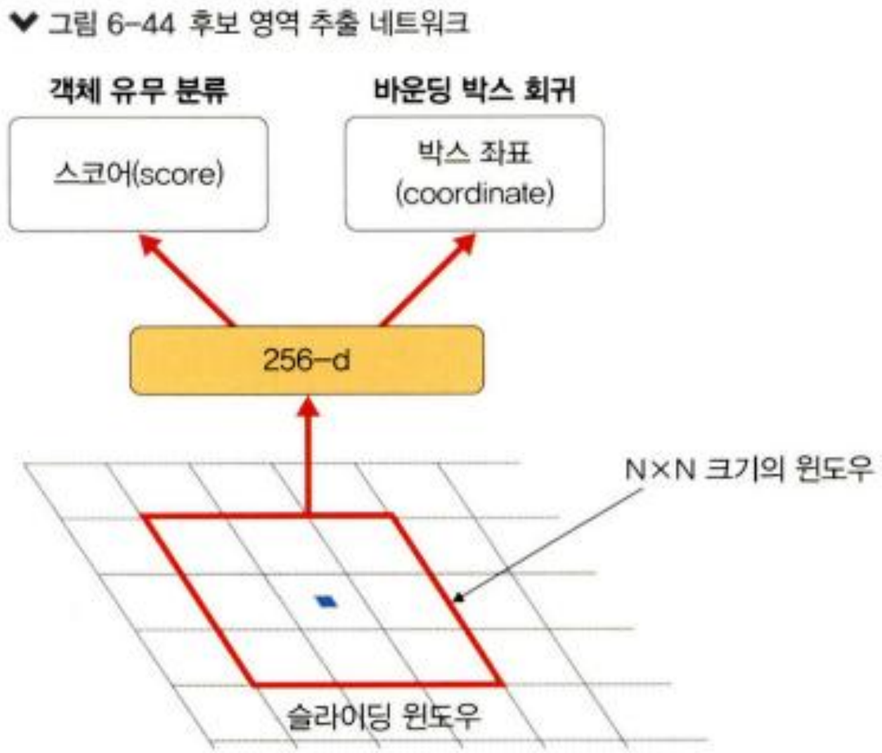


RPN은 CNN feature map을 입력으로 받아 후보 영역 생성
➔ RoI Pooling -> 분류기 -> bounding box regression

EWHA EURON

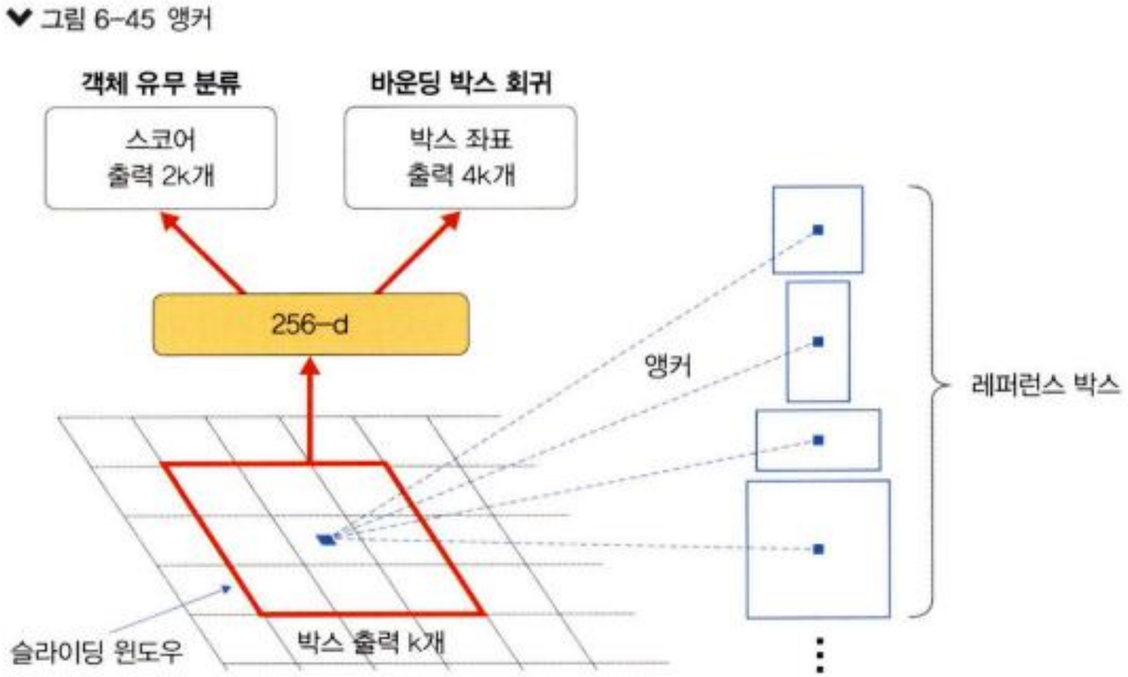
#07. RPN & Anchor

RPN: feature map 위 N×N sliding window -> 위치 확인
- 각 위치에서 객체/배경 이진 분류 + bounding box 위치 보정
- 고정된 N×N 영역만으로 다양한 객체 크기·비율을 수용하기 어려움



#07. RPN & Anchor

Anchor: 각 위치마다 크기·비율이 다른 reference box k개
- 출력: classification 2k, regression 4k / 전체 anchor = w×h×k



6.3 이미지 분할을 위한 신경망



EWHA EURON

#01. 이미지 분할

이미지 분할 : 이미지를 픽셀 단위로 분할
- 대표 네트워크 : FCN, Conv&Deconv, U-Net, PSPNet, DeepLabx3/v3+
- 객체의 정확한 형태/경계를 알아야 하는 문제에 적합
- 응용 예시 : 시멘틱 분할, 바이오 메디컬 이미지 분할

EWHA EURON

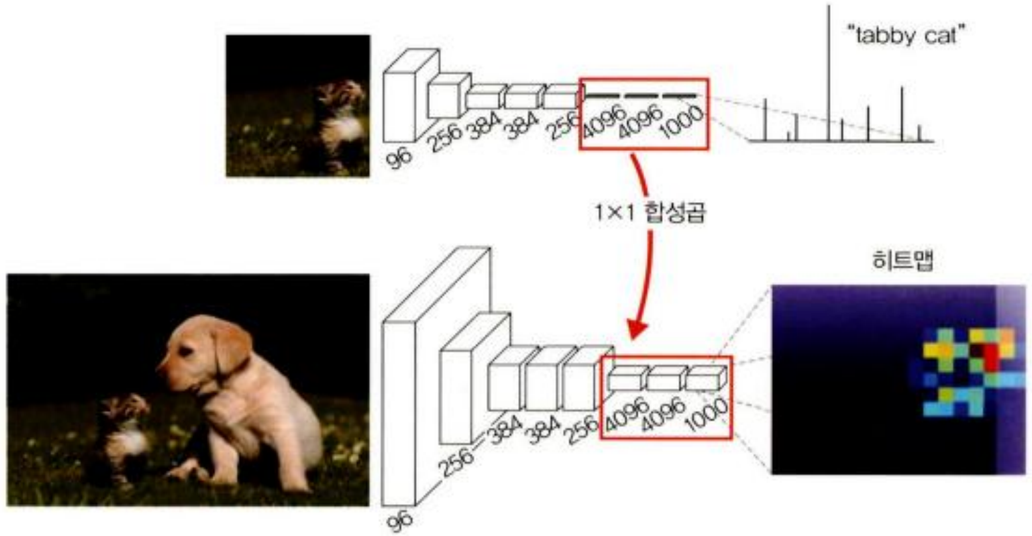
#02. FCN(완전 합성곱 네트워크)

기존 CNN의 마지막: 완전연결층
➔ 한계1. 고정된 입력크기
➔ 한계2. 위치 정보 소실

FCN : 완전연결층 -> 1x1 합성곱으로 대체

1x1합성곱 : 공간적 위치 정보 유지 + 입력 이미지 크기 제약 감소

그림 6-46 완전 합성곱 네트워크



#03. 합성곱 & 역합성곱

FCN : Conv/Pooling을 반복하면서 feature map 해상도 감소
but 최종 분할 결과는 입력 이미지 크기와 맞춰야함

역합성곱 : feature map 크기를 증가시키는 방향
- 교재 : 각 픽셀 주위에 zero-padding -> 합성곱 연산
=> 낮아진 해상도를 복원하는 업샘플링 역할

그림 6-47 합성곱 & 역합성곱 네트워크

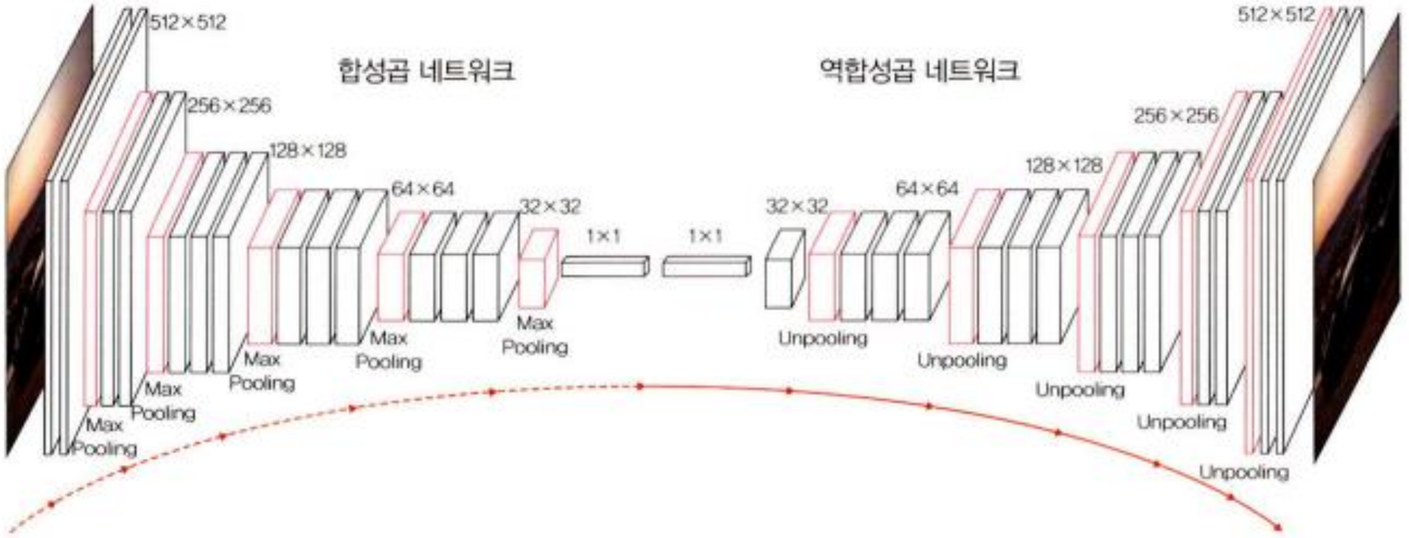
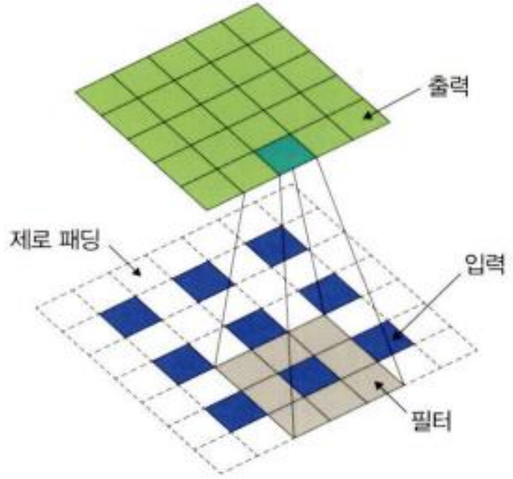
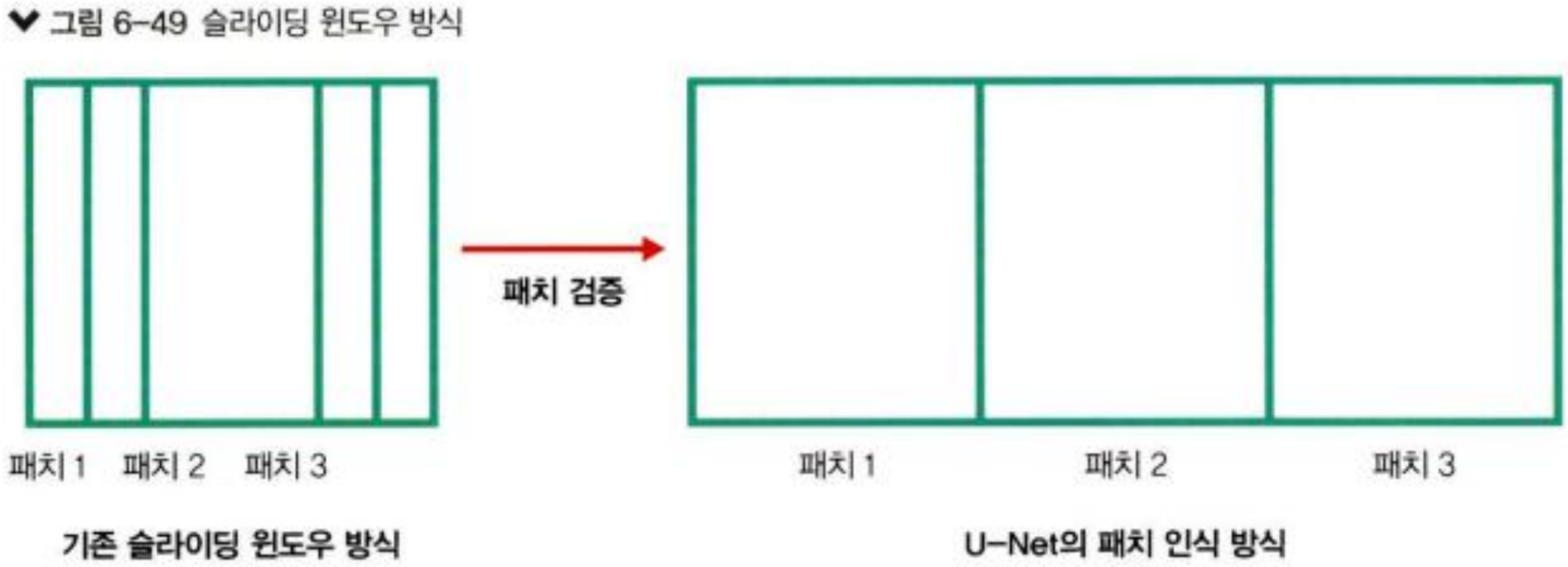


그림 6-48 역합성곱 진행 방식



#04. U-Net

U-Net : 바이오 메디컬 이미지 분할을 위한 CNN



큰 patch : context 인식 유리 / localization 불리
작은 patch : 넓은 context 부족/ localization 유리

U-Net은 context 인식과 localization 간 trade-off 개선

EWHA EURON

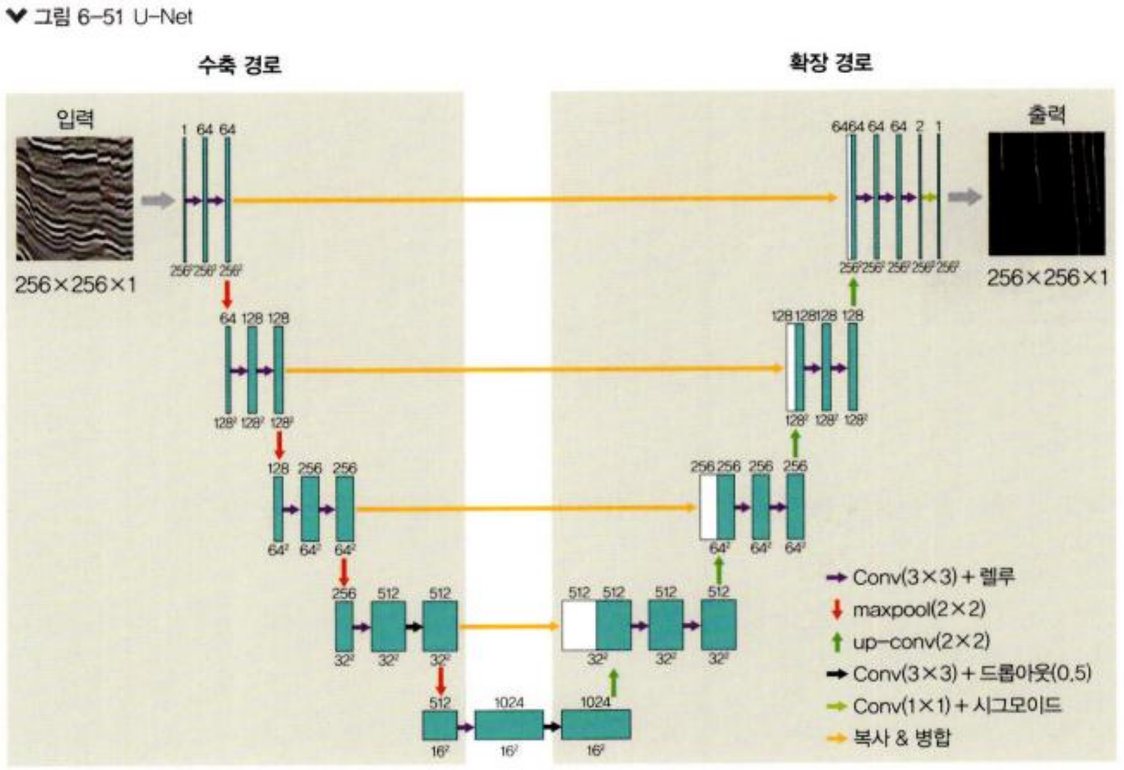
#05. U-Net의 구조

FCN 기반, U자 형태

수축 경로 : 3x3 conv 중심 + max pooling -> 크기 축소, context 포착
확장 경로 : up-conv -> 크기 커짐

확장 시 수축 경로의 feature map과 결합
➔ 세밀한 위치 정보 복원

다양한 크기의 feature map을 병합하여
정확한 localization

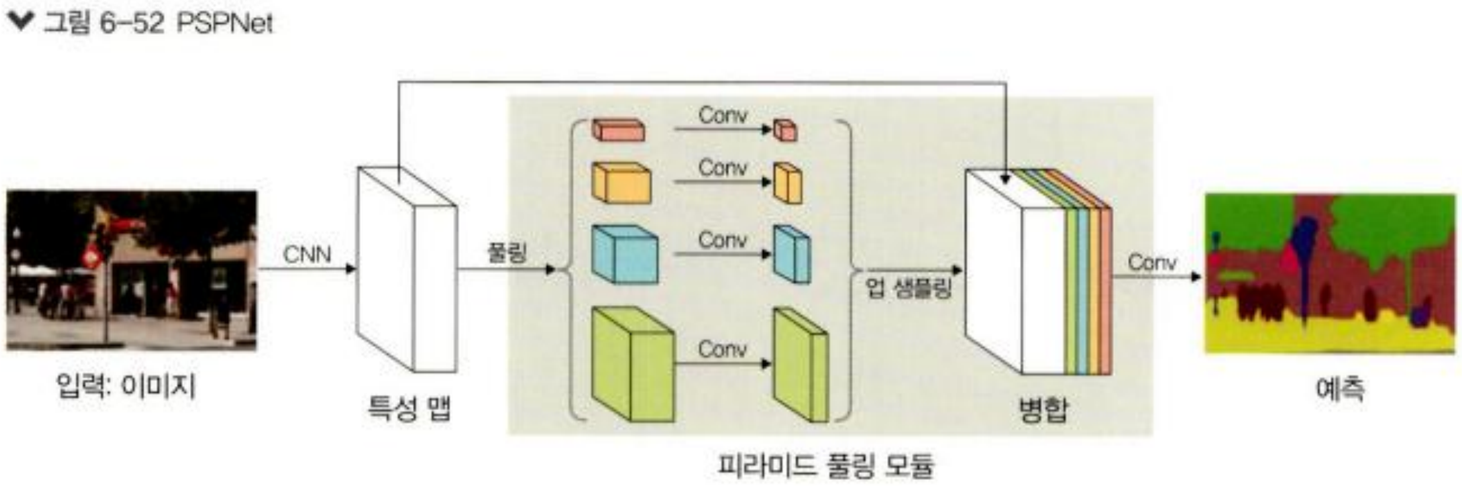


#06. PSPNet

Pyramid Pooling Module 추가

PSPNet 훈련 과정

1. 같은 특성 맵을 서로 다른 크기로 여러 번 풀링 - 서로 다른 범위의 정보 획득
2. 1x1 합성곱을 사용하여 채널 수 조정
   - 출력 채널 = 입력 채널/N
3. 모듈의 입력 크기에 맞게 특성 맵을 업 샘플링 – 양선형 보간법 사용
4. 원래의 특성 맵 + 새 특성 맵 병합

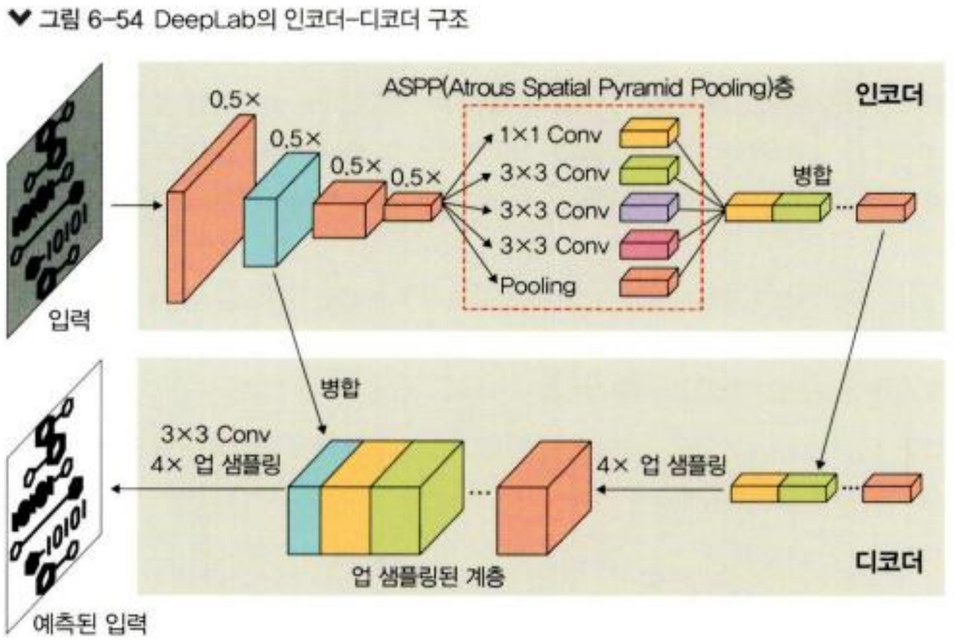


#07. DeepLabv3 / DeepLabv3+

DeepLabv3/v3+ : encoder-decoder 구조
- 특징 : Atrous 합성곱으로 encoder에서 추출된 feature map 해상도 제어

Atrous : filter 내부에 빈 공간을 둔 채 연산
ASPP(Atrous Spatial Pyramid Pooling) : 서로 다른 rate의 Atrous 합성곱을 이용해 여러 범위의 특징을 추출

rate가 빈 공간 크기 결정, r=1이면 일반 합성곱
r이 커질수록 더 넓은 범위를 볼 수 있음



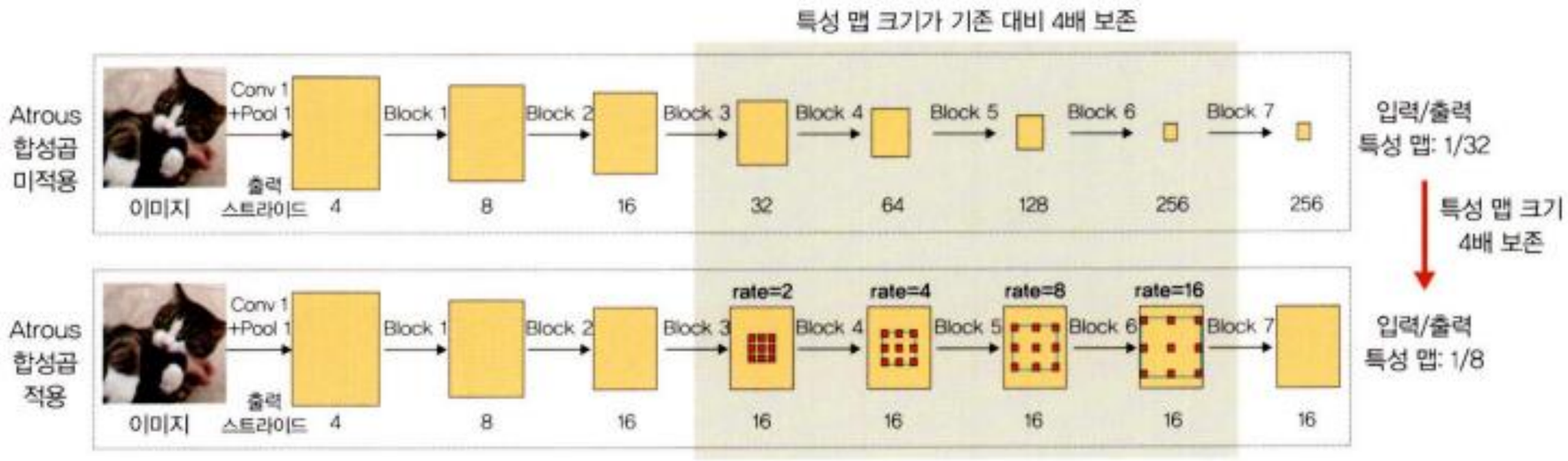
#08. Receptive Field

수용 영역(receptive field) : 한 출력 값이 영향을 받는 입력 범위
- 이미지 분할에서는 수용 영역 크기가 중요

Atrous conv : 파라미터 수를 늘리지 않으면서 수용 영역 확대

일반 CNN : 출력 약 1/32, Atrous 적용: 약 1/8
-> 한 변 기준 feature map 크기 4배 보존

❯ 그림 6-56 Atrous 합성곱 효과





6주차 질문

6장: 합성곱 신경망 2 – part2

EWHA EURON

질문1: 윤정

Q. 코드 6-83의 calculate_topk_accuracy 함수는 기본값 k=2로 상위 2개 예측 안에 정답이 있는지를 계산하고, 코드 6-84에서는 이 값을 acc_5라는 이름으로 받아 누적합니다. 그런데 코드 6-87의 학습 결과를 보면 10에포크 내내 Acc @5는 100.00%였고, Acc @1은 47~58%, loss는 반반으로 찍는 수준인 0.693 근처에 머물렀습니다.

개와 고양이 두 클래스뿐이라 상위 2개를 보면 모든 클래스를 다 보는 셈이니, 모델이 아무것도 학습하지 않아도 이 지표는 항상 100%가 나오는 것 같습니다. 교재 주석에도 “정확도가 100%일 것”이라고 되어 있는데, 이는 모델이 잘해서가 아니라 측정 방식 때문이라고 생각합니다. 또 변수 이름은 acc_5인데 실제로는 k=2로 계산되어서, 결과만 보면 “top-5 정확도 100%”로 오해할 수 있었습니다.

그래서 궁금한 점은, 지표가 높게 나왔을 때 그것이 진짜 성능인지 측정 방식 때문인지 구분하려면 무엇을 확인해야 할까요? 또한 top-k 정확도는 클래스 수에 비해 k가 어느 정도일 때 의미 있는 지표가 되는지 궁금합니다.

EWHA EURON

질문2: 조현아

Q. 6.3.2 합성곱 & 역합성곱 네트워크 부분의 도입에서는 완전 합성곱 네트워크의 단점으로 “낮아진 해상도를 복원하기 위해 업 샘플링 방식을 사용함 → 이미지의 세부 정보들을 잃어버리는 문제가 발생함”이라고 이야기합니다.

하지만 이후 역합성곱에 대해 설명하며 역합성곱을 업 샘플링이라고 이야기할 수 있다고 설명합니다. 문제의 원인과 해결책이 같은 단어가 혼용되고 있는 것 같은데, 두 단어는 각각 무슨 의미이고 어떤 역할을 하는지 궁금합니다.

EWHA EURON

논의: 이윤지

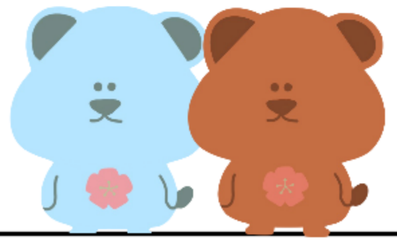
Q. ResNet과 U-Net은 모두 건너뛰기 연결을 사용하지만, ResNet은 입력과 합성곱층을 통과한 특징을 **더하고**, U-Net은 수축 경로와 확장 경로의 특징을 채널 방향으로 **연결**합니다.

책에서는 각 모델의 구조와 결합 방식을 설명하는데, 각 모델이 해결하려는 문제를 고려했을 때 왜 서로 다른 결합 방식을 사용하는지, 그리고 이 차이가 기존 특징을 유지하고 재활용하는 데 어떤 영향을 주는지 같이 추론해보고 싶습니다.

나아가 두 모델의 결합 방식을 서로 바꾼다면 특징 맵의 크기와 채널 수를 어떻게 조정해야 하는지, 그에 따라 학습과 예측 성능, 연산량과 메모리 사용은 어떻게 달라질지도 논의해보고 싶습니다.

EWHA EURON

THANK YOU



EWHA EURON